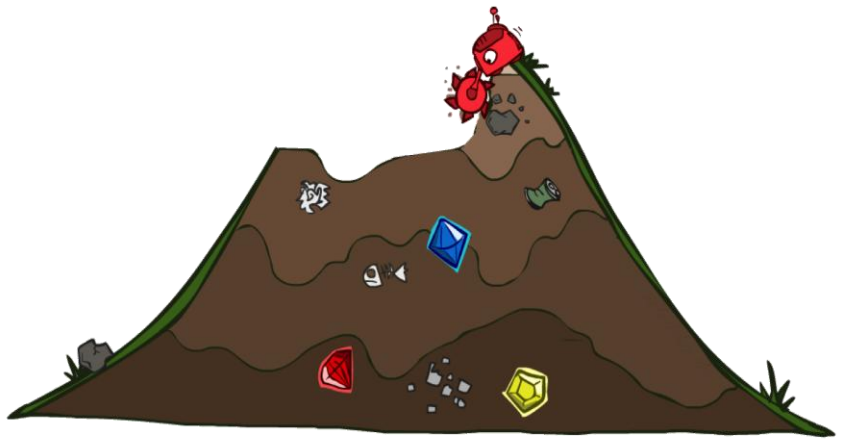
Artificial Intelligence

Informed (Heuristic) Search Strategies



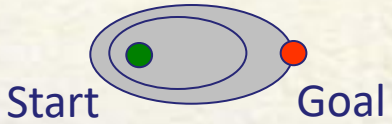
Uninformed vs Informed Search



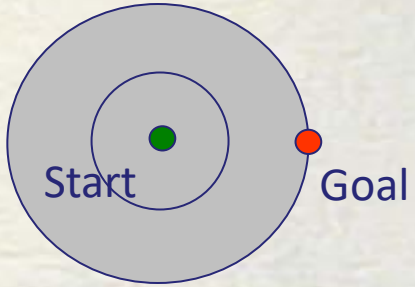


Effect of heuristics

Guide search ***towards the goal*** instead of ***all over the place***



Informed



Uninformed

Content

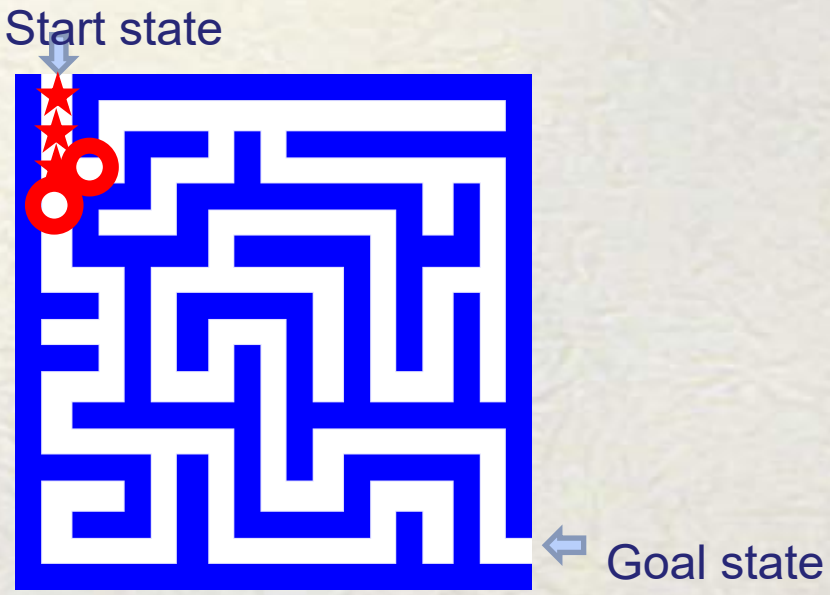
- Heuristic function
- Inform Search
  - Greedy Best-first search
  - Beam Search
  - Hill-climbing Search (Depth Best-First Search)
  - A* search

Heuristics (Kinh nghiệm)

- “Heuristics are criteria, methods or principles for deciding which among several alternative courses of action promises to be the most effective in order to achieve some goal.”
- Can make use of heuristics in deciding which is the most “promising” path to take during search.
- Evaluation function h(u): a measure to evaluate the *distance* of state u from the goal. e.g: h(u) = 0 if u is the goal state.
- Evaluation functions (or heuristic functions) are problem specific functions that provide an estimate of solution cost.

Search heuristics: estimates of distance-to-goal

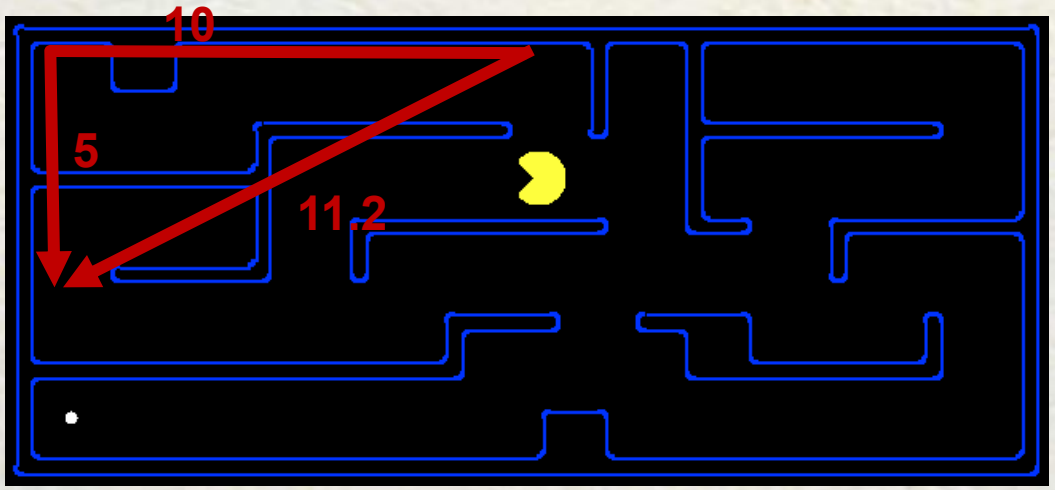
- Often, even if we don't know the distance to the goal, we can estimate it.
- This estimate is called a heuristic.
- A heuristic is useful if:
  1. **Accurate**: $h(n) \approx d(n)$, where $h(n)$ is the heuristic estimate, and $d(n)$ is the true distance to the goal
  2. **Cheap:** It can be computed in complexity less than $O\{b^d\}$

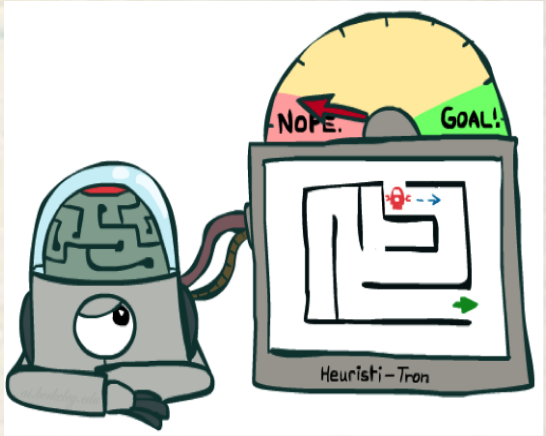


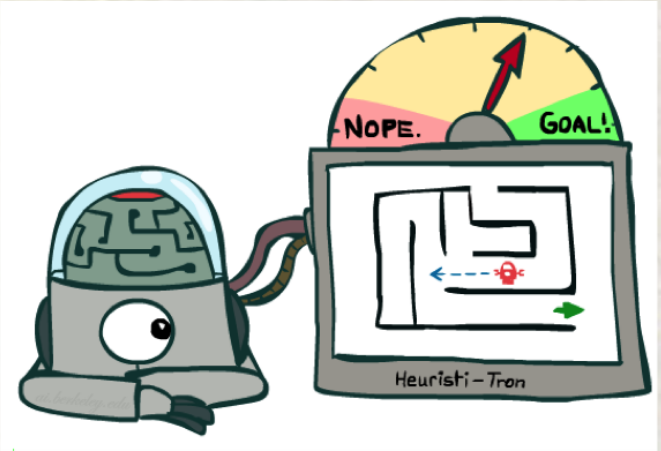
Search Heuristics

A heuristic function is:

- A function that *estimates* how close a state is to a goal
- Designed for a particular search problem
- Examples: Manhattan distance, Euclidean distance for pathing







Heuristic function

The 8-puzzle problem:

- Number of misplaced tiles, or
- Total Manhattan distance (number of squares from desired location of each tile)

4	3	1
	6	5
8	2	7

1	2	3
8		4
7	6	5

Heuristic function

- Number of misplaced tiles : 8

- Total Manhattan distance (number of squares from desired location of each tile) : 3 + 1 + 2 + 1 + 1 + 1 + 1 + 2 + 2 = 14

4	3	1
	6	5
8	2	7

1	2	3
8		4
7	6	5

Heuristic function

- There can be many ways to evaluate
- Evaluation functions may not be optimal
- How the evaluation function is chosen determines a lot of the results of heuristics search

Heuristics Search

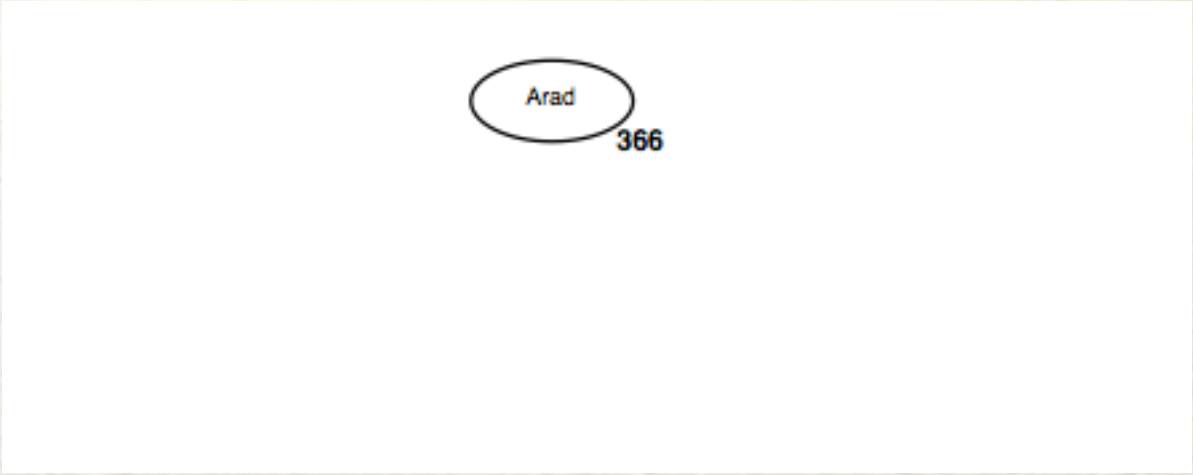
Three phases:

1. Find an appropriate representation describing the states and algorithms of the problem
2. Build the evaluation function
3. Design a strategy to choose a state to expand at each step (heuristic search)

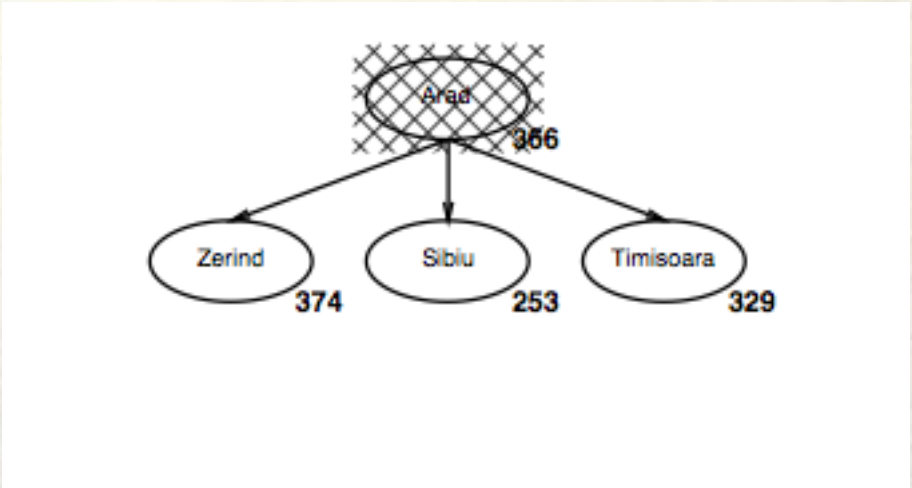
Greedy Best-first search

- Greedy Best-first search = Breadth first search + Heuristic function
- Unlike breadth-first search, Greedy Best-first search chooses the node to expand as the best node determined by the evaluation function.
- The best node is the one with the smallest evaluation function value, which can be at the current level or at levels above.

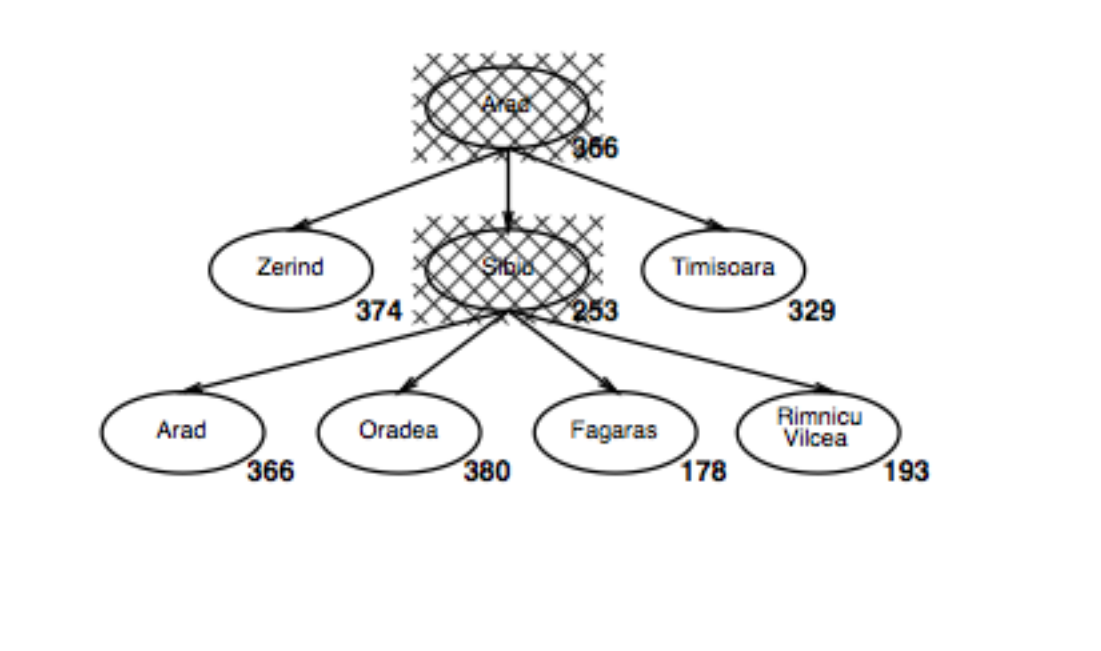
Greedy Best-first search (Ex.)



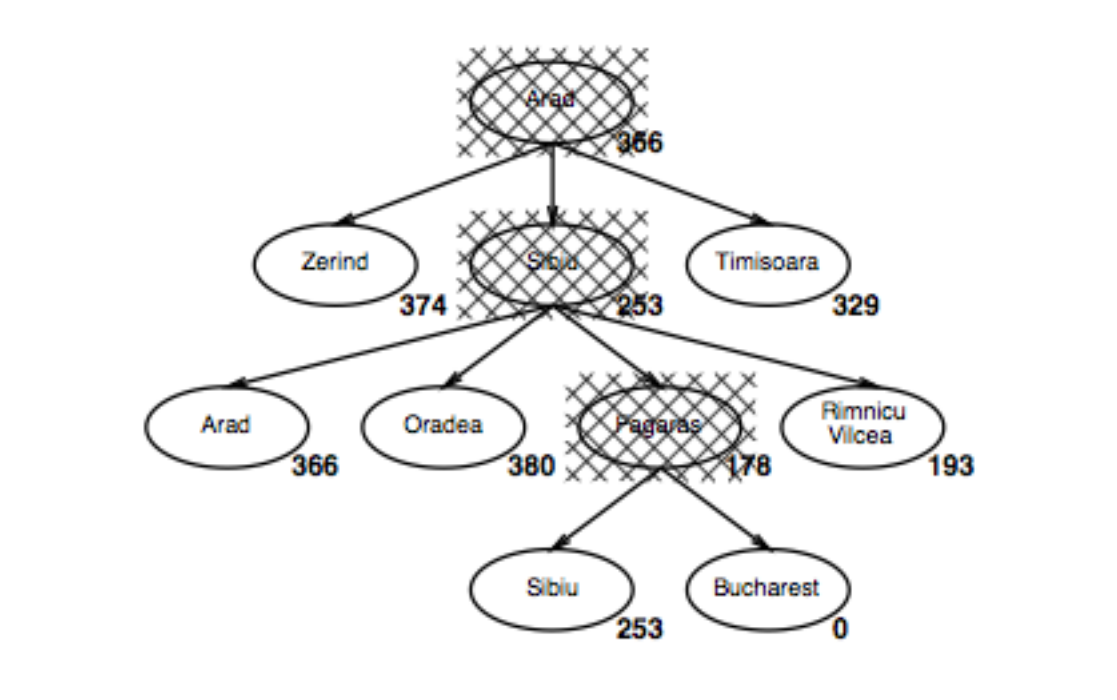
Example



Example


Arad
366
Zerind
374
Sibiu
253
Timisoara
329
Arad
366
Oradea
380
Fagaras
178
Rimnicu
Vilcea
193

Example

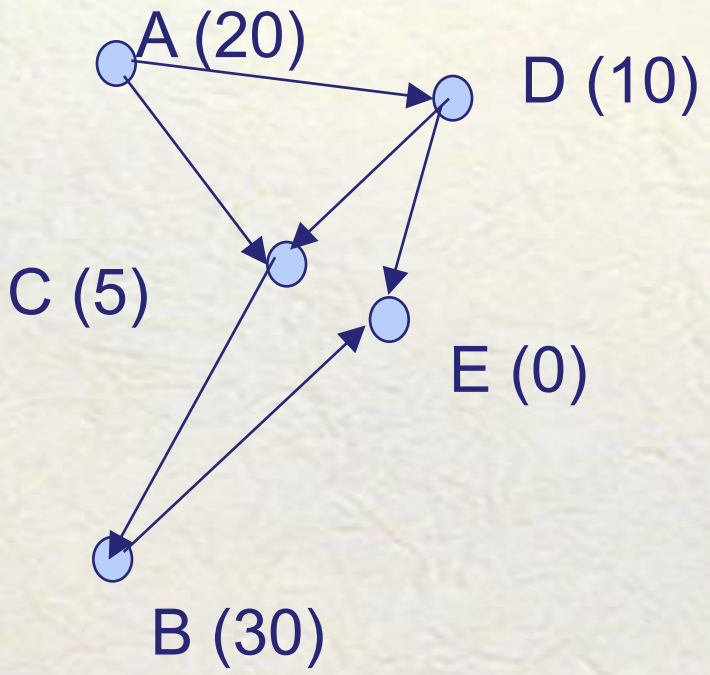

Arad
366
Zerind
374
Sibiu
253
Timisoara
329
Arad
366
Oradea
380
Fagaras
178
Rimnicu Vilcea
193
Sibiu
253
Bucharest
0

Greedy Best-first search

1. Initialize queue L containing only start state
2. **Loop do**
   - 2.1 **If** (L == Empty) **then**
     {failed search message; end}
   - 2.2 Remove u state from beginning of the **L**;
   - 2.3 **If** (u == Goal state) **then**
     {successful search message; end}
   - 2.4 **For** (each state v expanding from u) **do**
     {Put v in the list L so that L is sorted in best to worse order of the evaluation function}

Greedy Best-first search (Ex1)

Find path from A to E



- Find E

- **L**: A - Let A
- **L**: C, D - Let C
- **L**: D, B - Let D
- **L**: E, B - Let E
- Found E

Properties of Greedy best-first search

- Complete? **No** – can get stuck in loops, e.g., Iasi → Neamt → Iasi → Neamt →

  Complete in finite space with repeated-state checking
- Time? $O(b^m)$, m is the maximum depth in search space
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? **No**

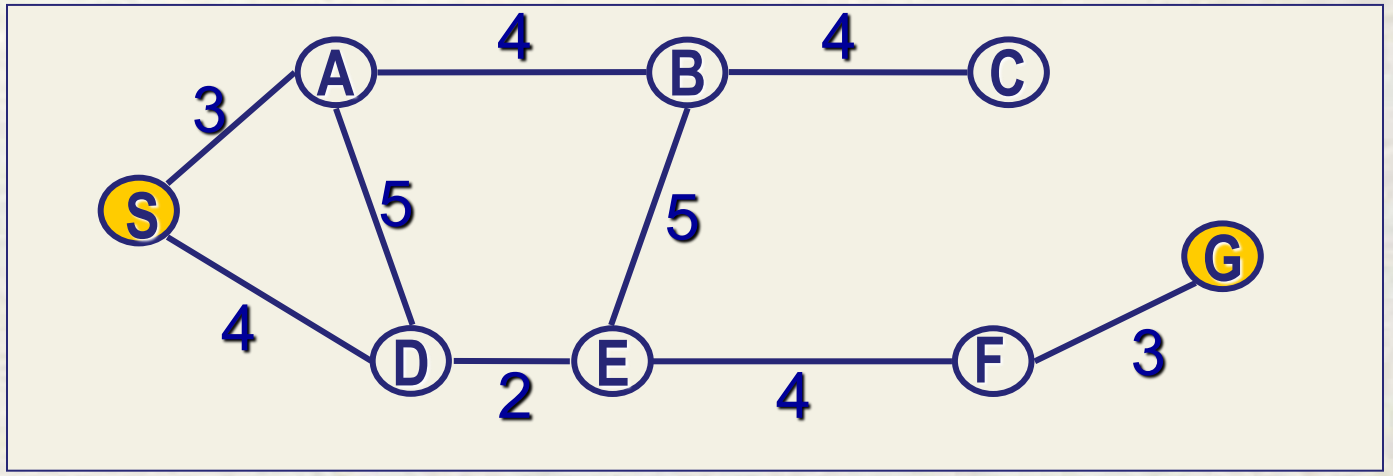
A good evaluation function can reduce memory time and space significantly.

Beam Search

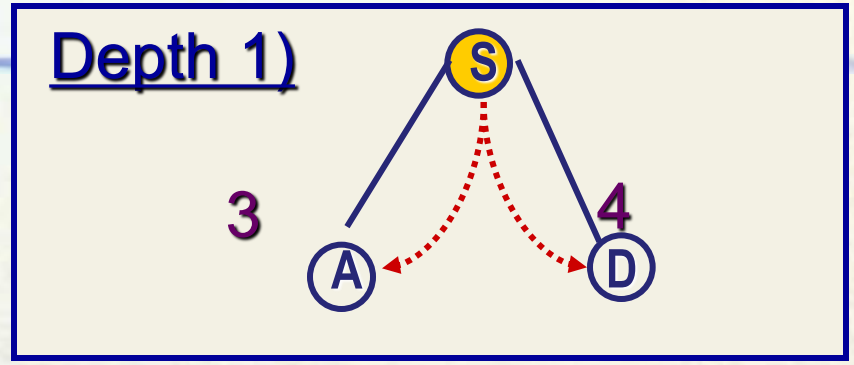
- Similar to best first search
- However, expand only k nodes at the next level, not the whole expands.
- Advantages: better computational complexity
- Disadvantage: does not search all, so the best nodes (goal nodes) may not be found.

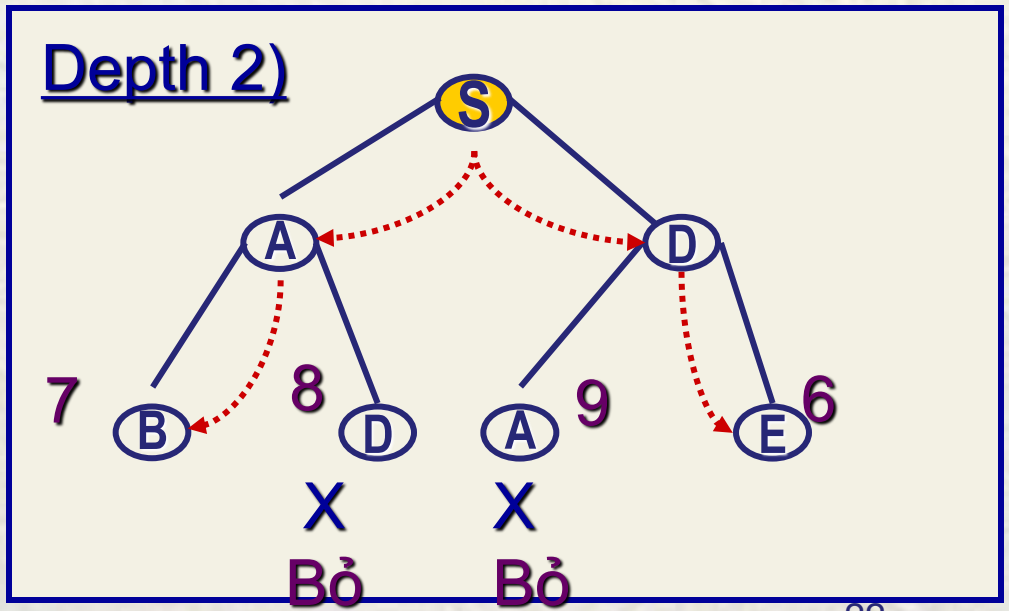
Example

- Imagine the problem of finding a route on a road map and that the NET below is the road map:



Beam search (1):



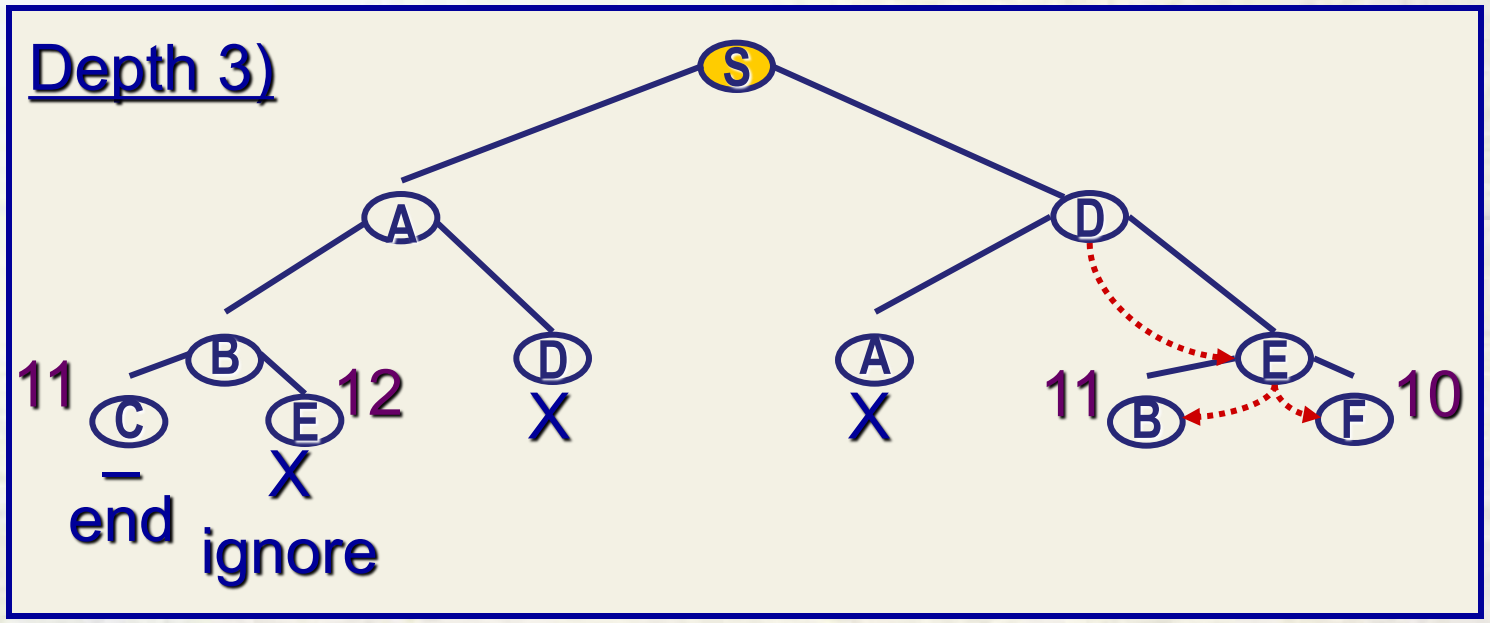


- Assume a pre-fixed WIDTH (example : 2)
- Perform breadth-first, BUT:
- Only keep the WIDTH best new nodes

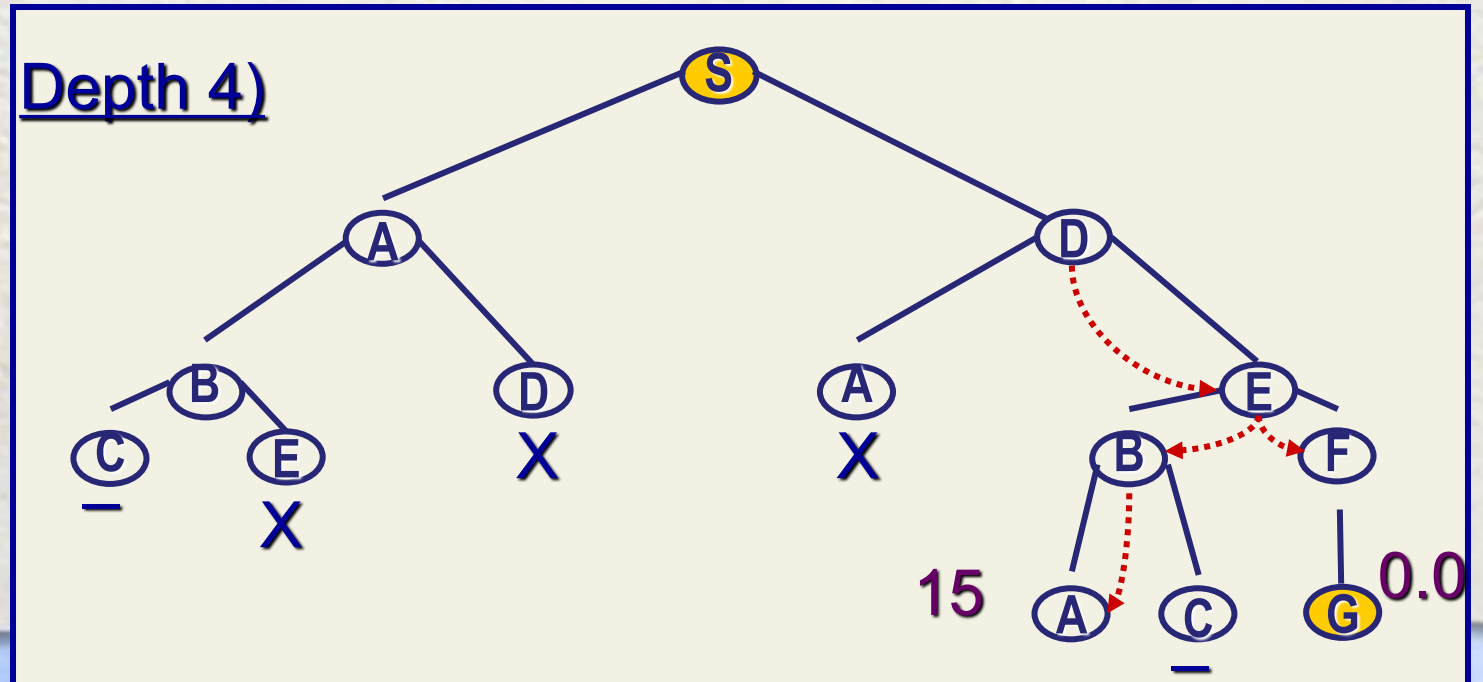
  depending on heuristic

  at each new level.

Beam search (2):



- Optimi-zation: ignore leafs that are not goal nodes



Beam search algorithm:

- **Homework**

Properties:

◎ **Completeness:**

➔ Beam search: NO

◎ **Speed/Memory:**

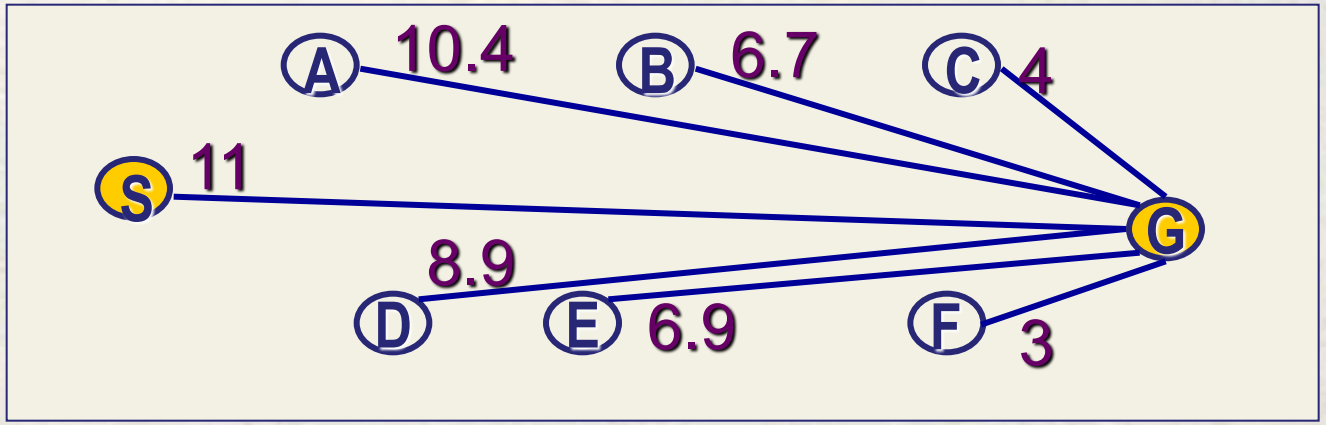
◆Beam search:

◆ QUEUE always has length WIDTH, so memory usage is constant = WIDTH, time is of the order of WIDTH*m*b or WIDTH*d*b if no solution is found

Hill-climbing Search

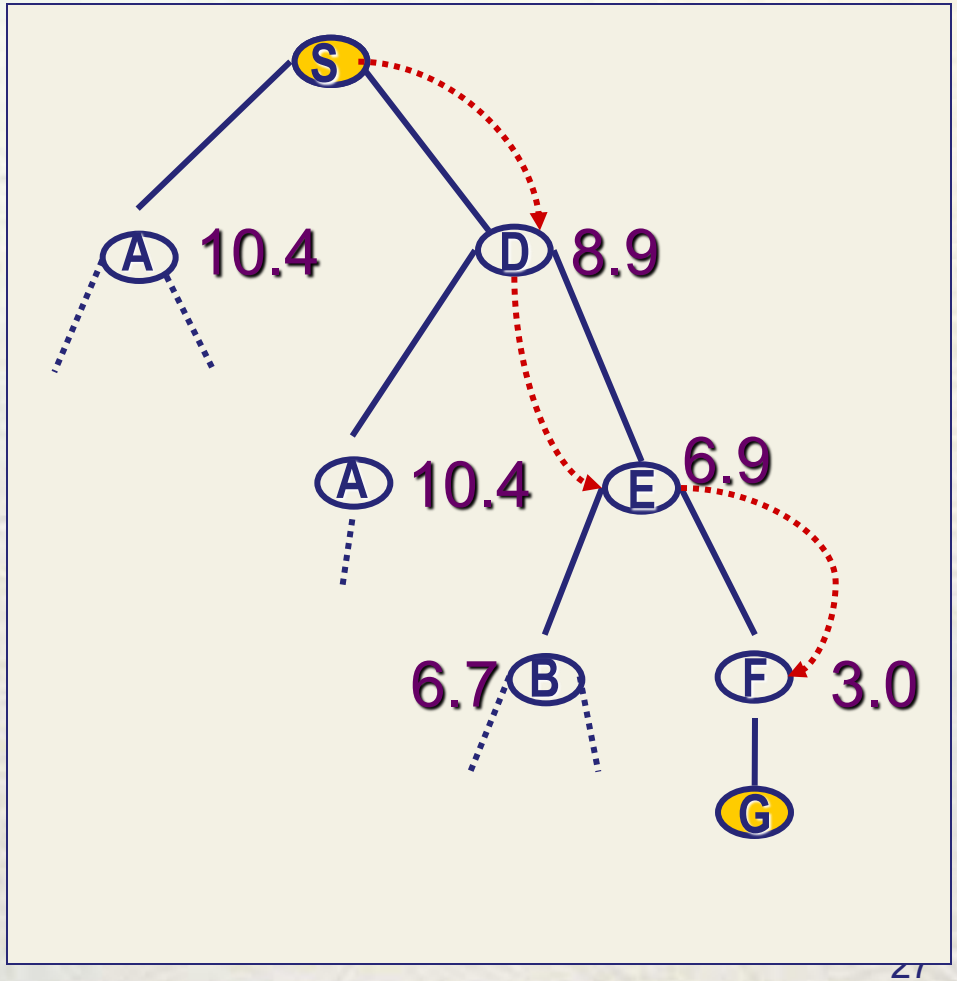
- Hill-climbing Search= Depth-first search + Heuristic function
- Unlike depth-first search, where we expand from node u, in the next step, we choose from among the expanded nodes of u, the node with the most promise to expand.
- The most promising node is the one with the smallest evaluation function

◎ Define f(T) = the straight-line distance from T to G



Hill-climbing Search

◎ Example: using the straight-line distance:



- Perform depth-first, BUT:
- instead of left-to-right selection,
- FIRST select the child with the best heuristic value

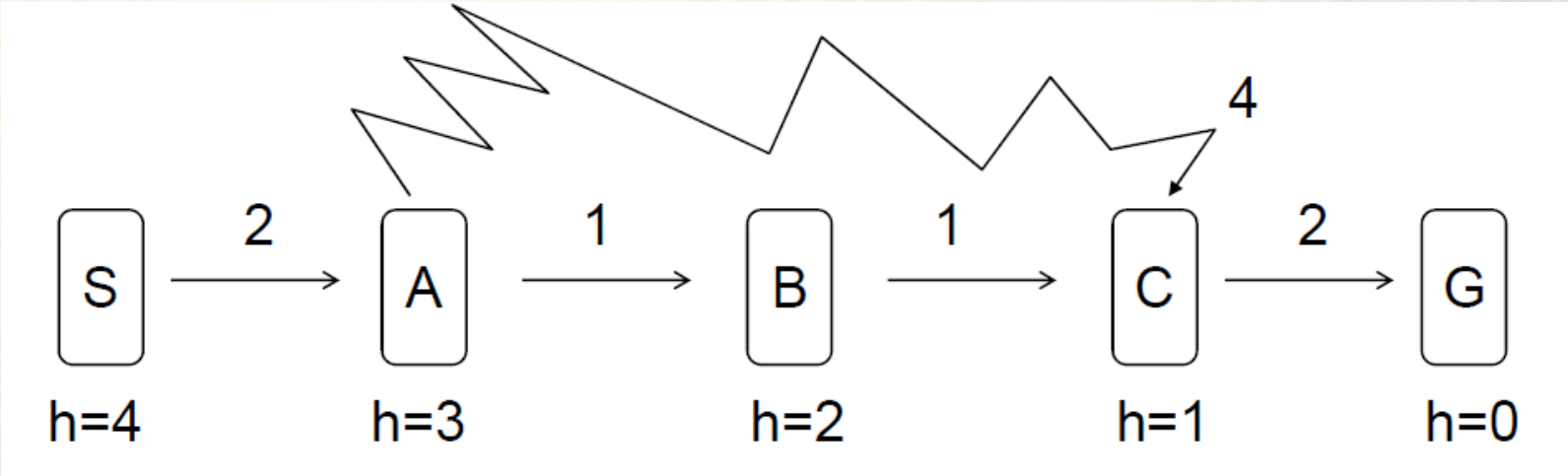
Hill-climbing Algorithm

1. Initialize queue L containing only start state;
2. **Loop do**
   2.1. **if** (L == Empty) then
   failed search message; end};
   2.2. Remove u state from beginning of the L;
   2.3. **if** (u == Goal state) **then**
   {successful search message; end};
   2.4. **for** (each state v expanding from u) do put v into **L1**;
   2.5. Sort L1 in ascending order of the evaluation function so that the best state is at the top of the L1
   2.6. Move L1 to the beginning of L so that the beginning of L1 becomes the beginning of L

Properties of Hill-climbing search

- **Completeness:**
  - Hill climbing: YES (backtracking)
- **Speed/Memory:**
  - same as Depth-first (in worst case)
- **Optimal**
  - No

How worse “Greedy Best First Search” ?



- Best First Search guarantees the optimal path?
- How can we fix the greedy problem?

A* Search

A* Search



UCS



Greedy



A*

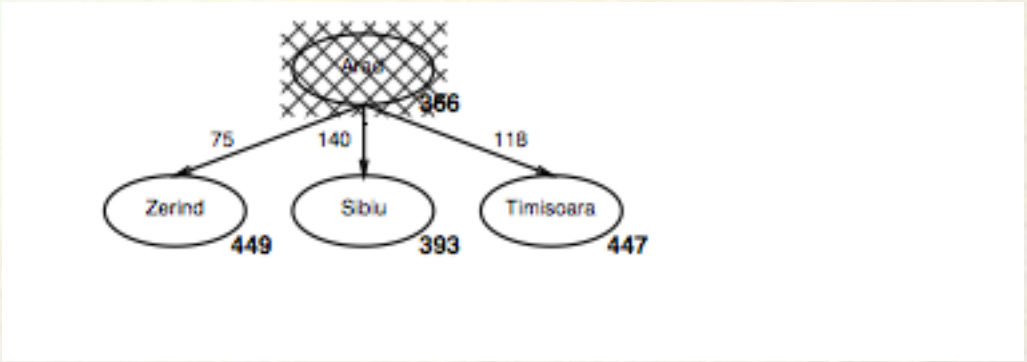
A* Search

- A* Search uses evaluation function f(n) = g(n) + h(n)
  - g(n): cost from initial node to node n
  - h(n): estimated cost of cheapest path from n to goal.
  - f(n): estimated total cost of cheapest solution through n.
- Best first (Greedy) Search minimizes h(n)
  - Effective but not optimal
- Uniform-cost Search minimizes g(n)
  - Optimal but not effective

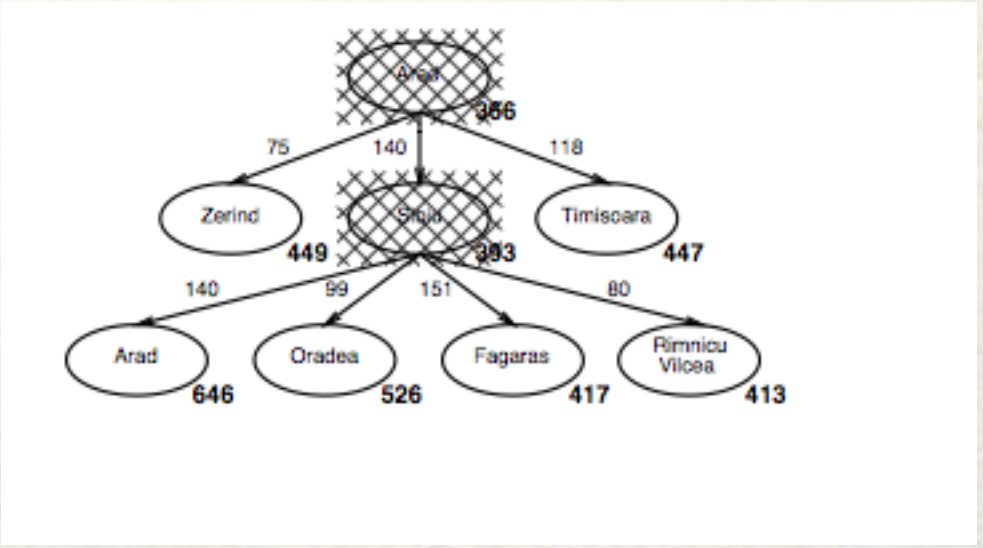
A* Search Example



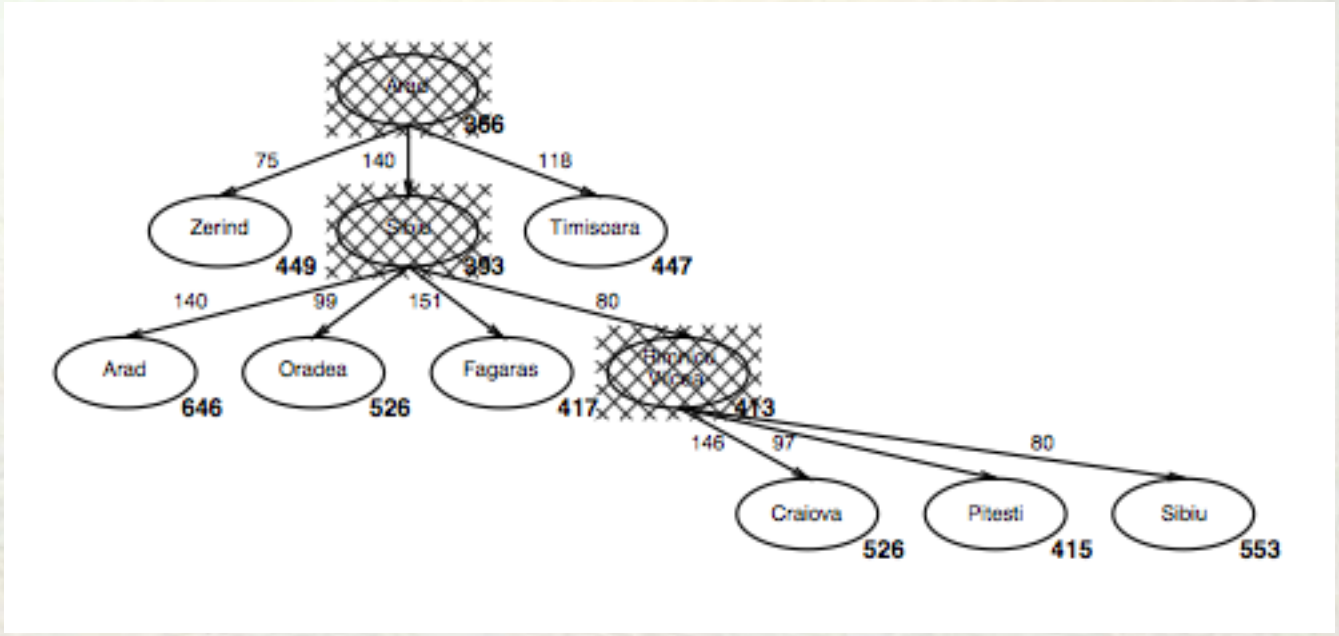
A* Search Example



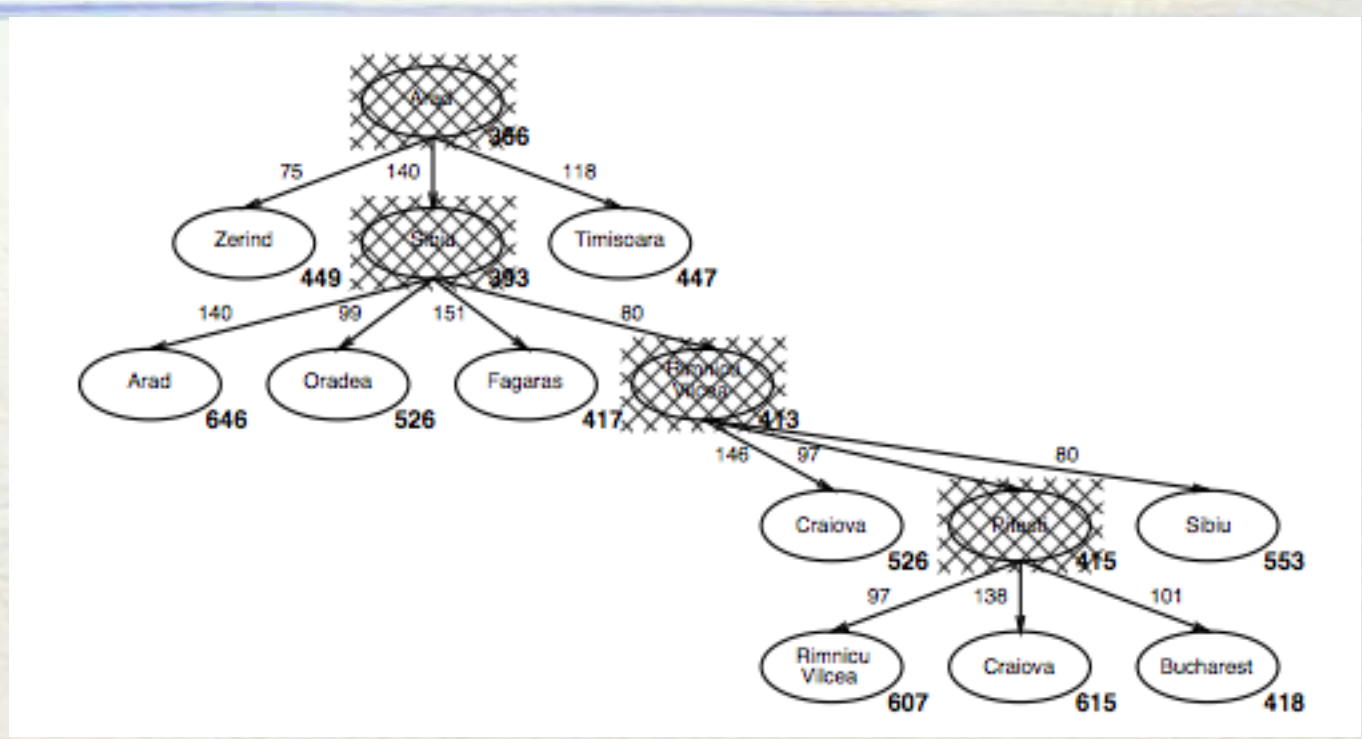
A* Search Example



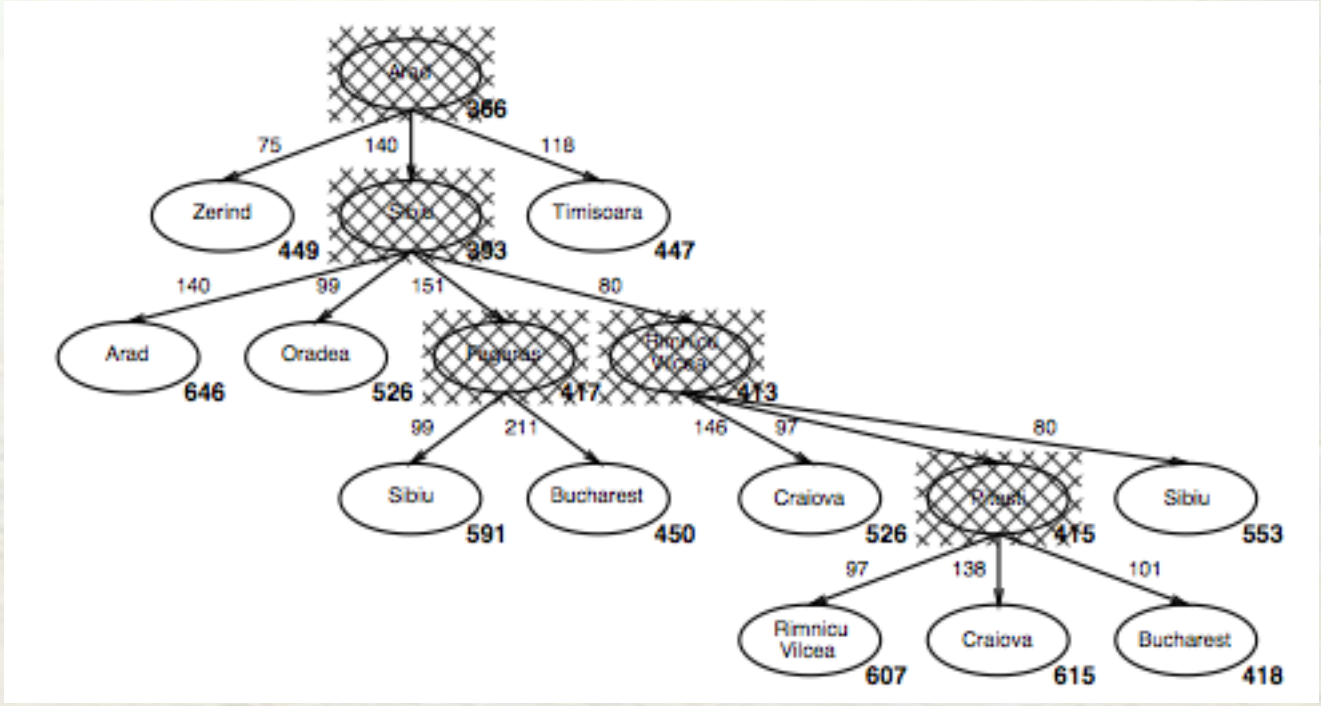
A* Search Example



A* Search Example



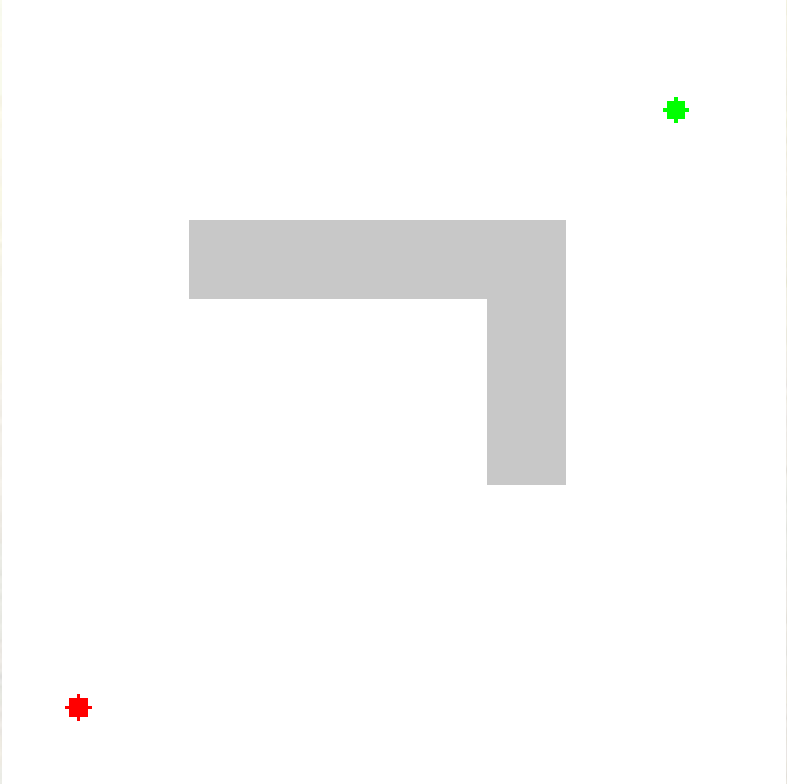
A* Search Example



A* Algorithm

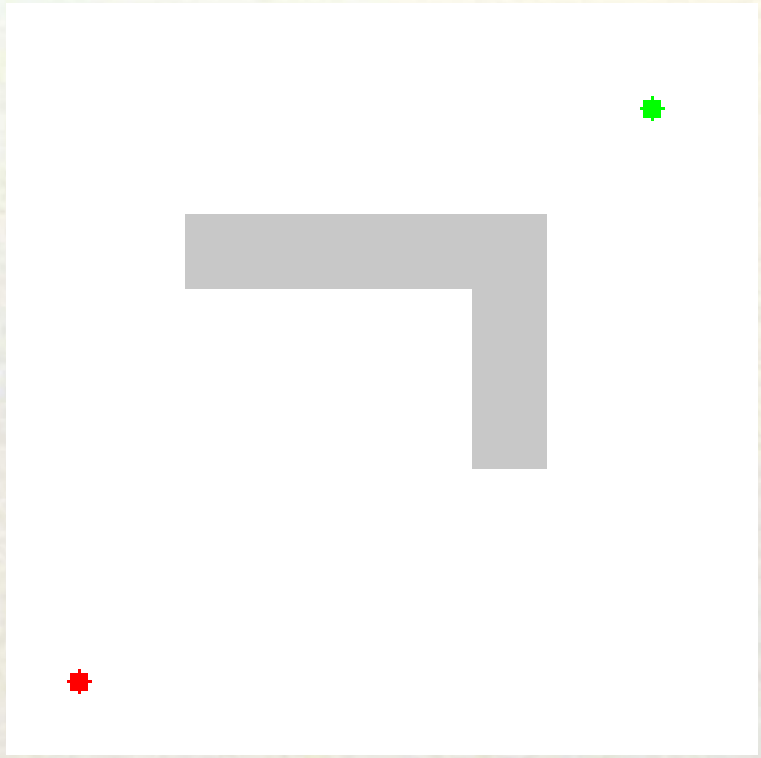
1. Initialize queue L containing only start state
2. **Loop do**
   - 2.1 **If** (L == Empty) **then**
     {failed search message; end}
   - 2.2 Remove u state from beginning of the queue L;;
   - 2.3 **If** (u == Goal state) **then**
     {successful search message; end}
   - 2.4 **For** (each state v expands from u) **do**
     {g(v) := g(u) + k(u,v);
     f(v) := g(v) + h(v);
     Put v into the queue L so that L is sorted in best to worse order of the evaluation function;}

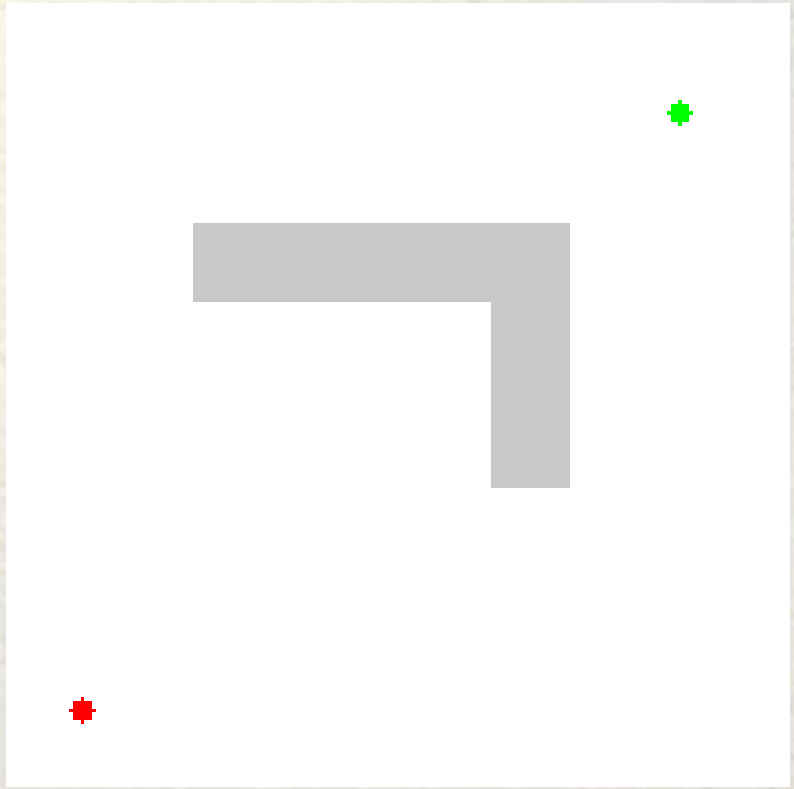
Another example



Source: Wikipedia

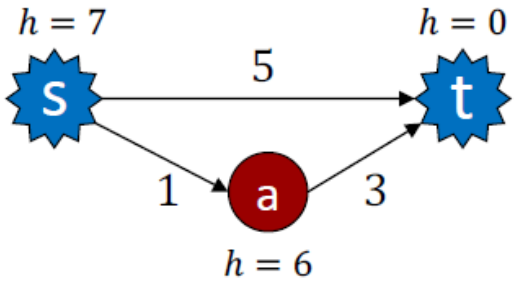
Uniform cost search vs. A* search





Source: Wikipedia

Is A* Optimal?



$$F = \{a, t\}, \qquad f(a) = 1 + 6 = 7, \qquad f(t) = 5 + 0$$

Expand t, goal check = true → end
Optimal path not discovered! ☹



What's the problem?

A* Search

- A* Search makes f(n) = g(n) + h(n) minimize.
  - **Idea**: preserve efficiency of Greedy Search but avoid expanding path that are already expensive
- Question: Is A* search optimal and complete?
- Yes! Provided h(n) is *admissible*- if h(n) does not exceed the actual value from state n to the goal state.
- http://theory.stanford.edu/~amitp/GameProgramming/AStarComparison.html

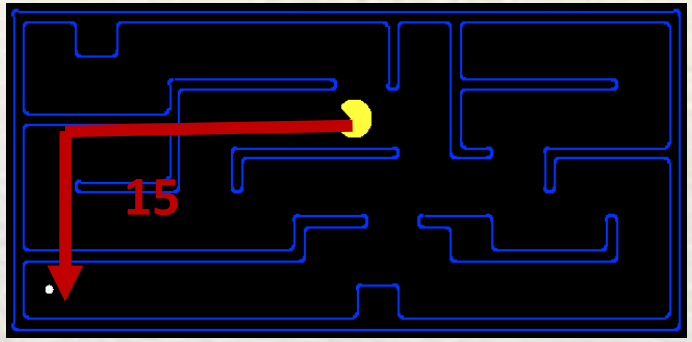
Admissible Heuristics

- A heuristic h is *admissible* () if:

  $$0 \leq h(n) \leq h^*(n)$$

  where $h^*(n)$ is the true cost to a nearest goal

- Example:

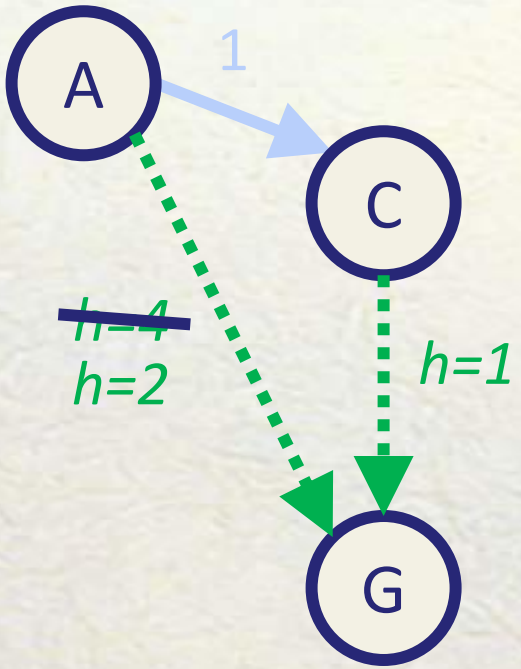


- With **admissible heuristics**, **A* search is optimal.**

Optimality of A* Search

- Even after a goal node has been generated, A* will keep searching so long as there is a possibility of finding a shorter solution.
- Once a goal node is selected for expansion, we know it must be optimal, so we can terminate the search.

Consistency of Heuristics



- Main idea: Estimated heuristic costs ≤ actual costs
  - Admissibility:

    heuristic cost ≤ actual cost to goal

    h(A) ≤ actual cost from A to G
  - Consistency:

    "heuristic step cost" ≤ actual cost for each step

    h(A) – h(C) ≤ cost(A to C)

    triangle inequality

    h(A) ≤ cost(A to C) + h(C)

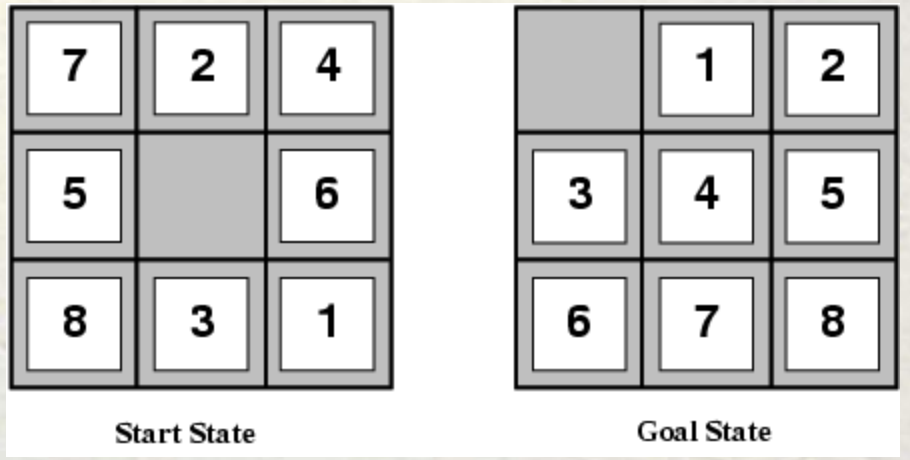
Properties of A* search

- Complete? Yes
- Time? Exponential
- Space? Keeps all nodes in memory
- Optimal? Yes
-

Designing heuristic functions

Heuristics for 8-puzzle problem:

- $h_1(n)$ = Number of misplaced tiles
- $h_2(n)$ = Total Manhattan distance (number of squares from desired location of each tile)

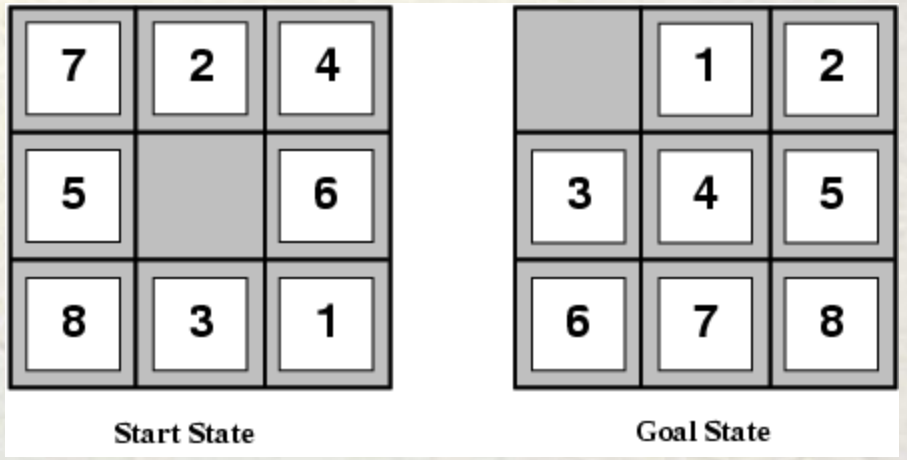


- $h_1(S) = ?$
- $h_2(S) = ?$
-

Designing heuristic functions

Heuristics for 8-puzzle problem :

- $h_1(n)$ = Number of misplaced tiles
- $h_2(n)$ = Total Manhattan distance (number of squares from desired location of each tile)



- $h_1(S) = ?$ 8
- $h_2(S) = ?$ 3+1+2+2+2+3+3+2 = 18

Dominance

- If $h_2(n) \geq h_1(n)$ for all n (both admissible)
- then h_2 dominates h_1 (h_2 *áp đảo* h_1)
- h_2 is better for search
-
- Typical search costs (average number of nodes expanded):

- *Depth d=12*

  IDS = 3,644,035 nodes
  $A^*(h_1)$ = 227 nodes
  $A^*(h_2)$ = 73 nodes
- *Depth d=24*

  IDS = too many nodes ~ $54 * 10^9$ nodes
  $A^*(h_1)$ = 39,135 nodes
  $A^*(h_2)$ = 1,641 nodes

Heuristics from relaxed problems

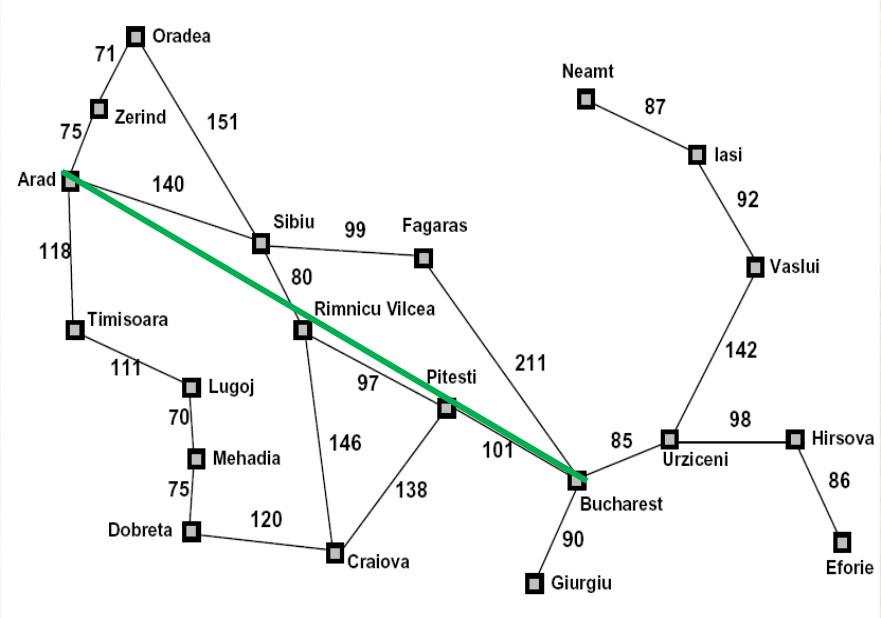
- Reduce constraints
- A problem with less constraints than in actions is called relaxed problem
- The value of the optimal solution for the relaxed problem is the admissible function of the original problem

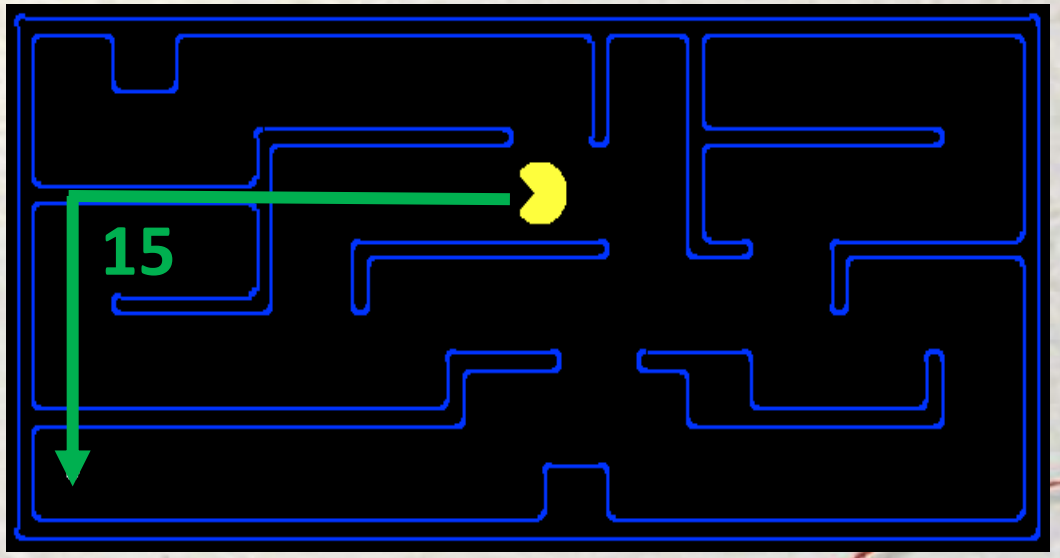
- If the rule of the 8-puzzle problem is relaxed so that the pieces can move freely (anywhere), then $h_1(n)$ gives the shortest solution.
- If the rule of the 8-puzzle problem is relaxed so that the pieces can move any adjacent squares, then $h_2(n)$ gives the shortest solution.

Creating Admissible Heuristics

- Most of the work in solving hard search problems optimally is in coming up with admissible heuristics
- Often, admissible heuristics are solutions to ***relaxed problems***, where new actions are available

366





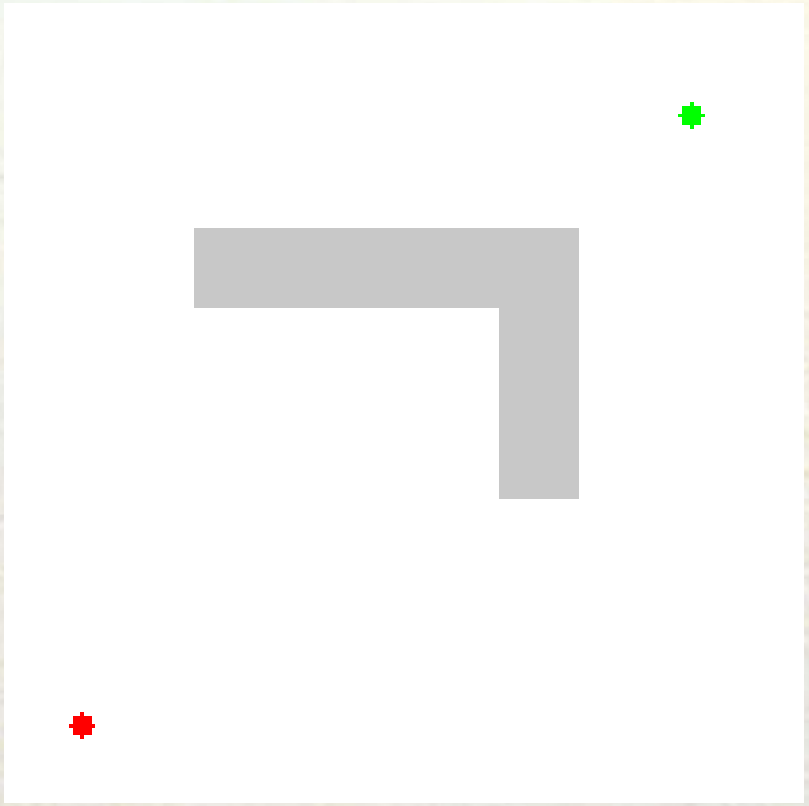
Composite Heuristic Functions

- Let h_1, h_2,.., h_m be admissible heuristics for a given task.
- Define the composite heuristic:
  - $h(n) = \max(h_1(n), h_2(n), \ldots, h_m(n))$.
- h is admissible heuristic
- h dominate h_1, h_2, …, h_m

Weighted A* search

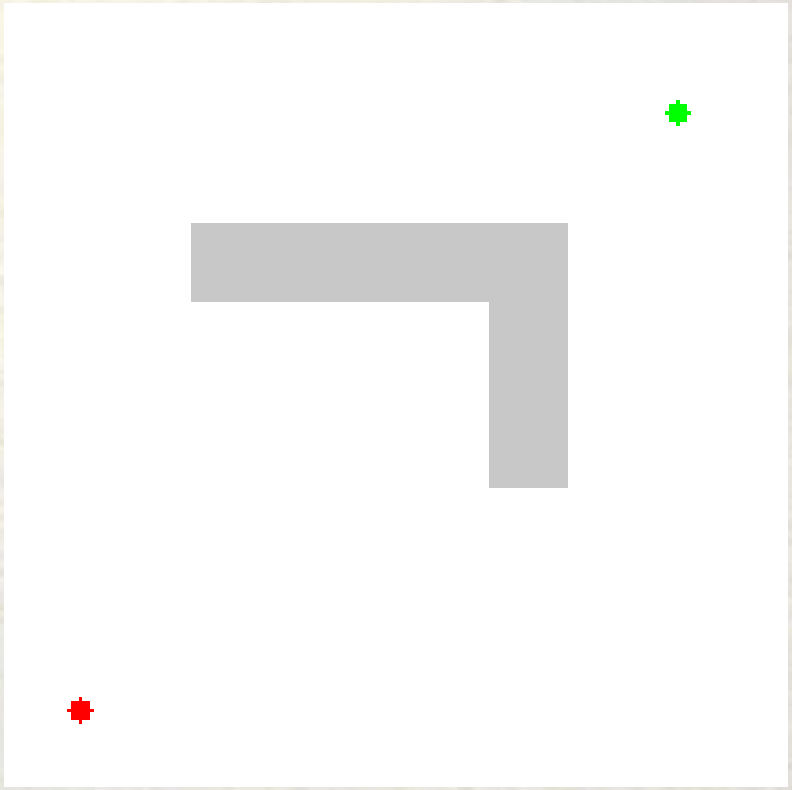
- **Idea:** speed up search at the expense of optimality
- Take an admissible heuristic, “inflate” it by a multiple $\alpha > 1$, and then perform A* search as usual
- Fewer nodes tend to get expanded, but the resulting solution may be suboptimal (its cost will be at most α times the cost of the optimal solution)

Example of weighted A* search



Heuristic: 5 * Euclidean distance from goal

Source: Wikipedia



Compare: Exact A*

A* Applications

- Pathing / routing problems
- Resource planning problems
- Robot motion planning
- Language analysis
- Video games
- Machine translation
- Speech recognition
- …

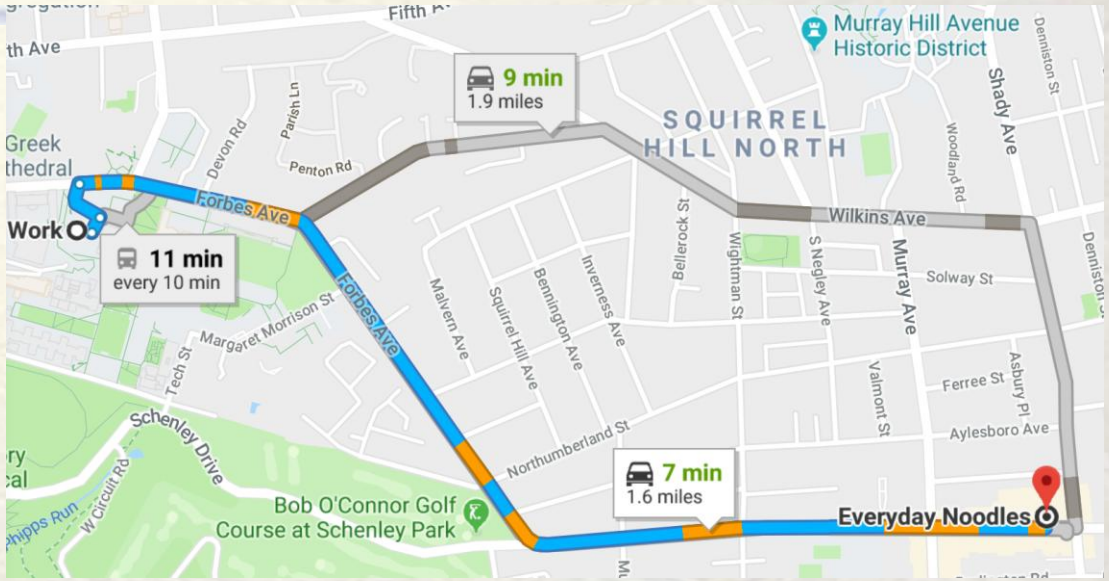
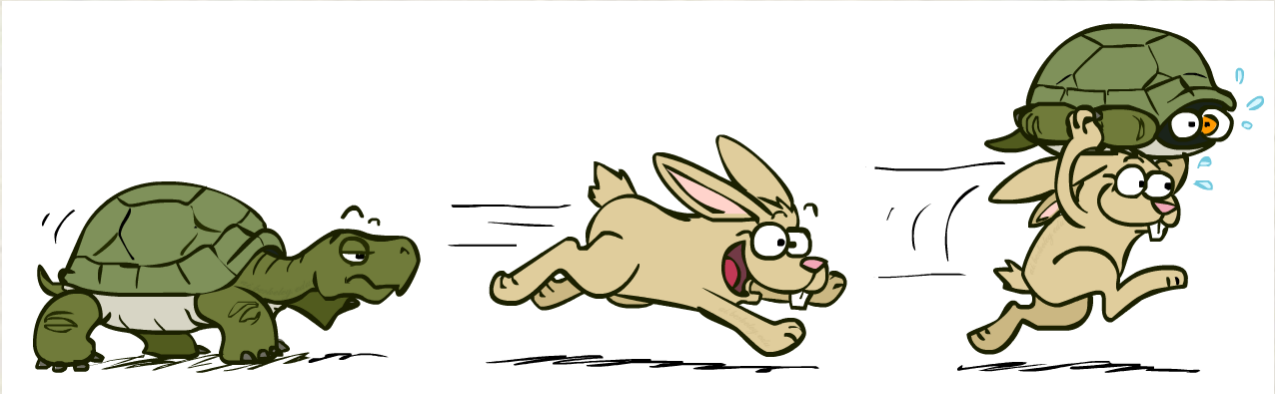


Image: maps.google.com

A*: Summary

- A* uses both backward costs and (estimates of) forward costs
- A* is optimal with admissible / consistent heuristics
- **Heuristic design is key**: often use relaxed problems



All search strategies. C*=cost of best path.

Algorithm	Complete?	Optimal?	Time complexity	Space complexity	Implement the Frontier as a…
BFS	Yes	If all step costs are equal	O(b^d)	O(b^d)	Queue
DFS	No	No	O(b^m)	O(bm)	Stack
IDS	Yes	If all step costs are equal	O(b^d)	O(bd)	Stack
UCS	Yes	Yes	Number of nodes w/ $g(n) \leq C^*$	Number of nodes w/ $g(n) \leq C^*$	Priority Queue sorted by g(n)
Greedy	No	No	Worst case: O(b^m) Best case: O(bd)	Worse case: O(b^m) Best case: O(bd)	Priority Queue sorted by h(n)
A*	Yes	Yes	Number of nodes w/ $g(n)+h(n) \leq C^*$	Number of nodes w/ $g(n)+h(n) \leq C^*$	Priority Queue sorted by h(n)+g(n)

Reference

- Trí Tuệ Nhân Tạo, Đinh Mạnh Tường
- Artificial Intelligence, A modern Approach. Chapter 4.

Find shortest path from S-> G by A* algorithm

